Requirement Specification: AutoTestDesign

AI App

1. Introduction

The **AutoTestDesign AI Application** is a proposed system designed to automate and enhance software test design activities, aligning with industry standards like **ISTQB Foundation Level** principles and the detailed test techniques defined in **ISO/IEC/IEEE 29119-4**. The primary goal is to leverage Artificial Intelligence (AI) and Machine Learning (ML) to perform requirements analysis, risk assessment, and systematic test case generation, thereby improving test efficiency and coverage.

2. Goals and Objectives

- **G1: Standard Alignment:** Ensure generated test artifacts (test cases, priority scores) comply with established testing standards (ISTQB terminology and ISO 29119 structure).
- **G2: Automation:** Automate complex, rule-based test design techniques (e.g., EP, BVA, Decision Tables).
- **G3: Risk-Based Testing:** Integrate risk analysis early in the process to prioritize testing effort.
- **G4: Modularity:** Design a platform that allows easy integration of new test design algorithms and AI models as distinct modules.

---

3. Functional Requirements (FR)

The system must support the following core features:

ID	Feature Category	Description
FR 1.0	**Input/Parsing**	The system must be able to **ingest software requirements** from various sources (e.g., CSV, plain text, or direct user input).
FR 1.1	**Requirement Structuring**	The system must use **Natural Language Processing (NLP)** to parse and tokenize raw text, identifying key components like **Input Fields**, **Data Ranges**, **Conditions**, and **Expected Actions**.

ID	Feature Category	Description
FR 2.0	**Risk Analysis & Prioritization**	The system must employ an **AI/ML model** (Project 1) to assign a **Risk Score** and **Test Priority** (High, Medium, Low) to each imported requirement.
FR 3.0	**Black-Box Test Design**	The system must include modules to automatically apply and generate test cases for at least three core Black-Box techniques from ISO 29119-4 (e.g., **Equivalence Partitioning**, **Boundary Value Analysis**, **Decision Tables**).
FR 4.0	**White-Box Test Modeling**	The system must include a module to model system behavior (e.g., **State Transition Diagram**) and generate optimal test sequences for a chosen coverage criterion (e.g., **All States**).
FR 5.0	**Test Oracle Generation**	The system must include a component (Project 10) to help **synthesize the Expected Result** for a given requirement and specific test data.
FR 6.0	**Output & Export**	The system must generate test artifacts (Test Cases, Test Suites, Risk Scores) in a structured, **standard format** (e.g., JSON or Excel/CSV) suitable for import into Test Management Tools.
FR 7.0	**Test Suite Optimization**	The system must include an optimization module (Project 4 or 9) to prioritize or minimize the generated Test Suite based on risk or coverage efficiency.

4. Non-Functional Requirements (NFR)

4.1. Performance

- **NFR 4.1.1 (Analysis Time):** Requirements processing and risk analysis for a batch of 100 requirements must complete within **5 seconds**.

- **NFR 4.1.2 (Test Case Generation):** Generation of a full set of test cases for a single requirement (e.g., Decision Table) must not exceed **2 seconds**.

4.2. Usability (UX/UI)

- **NFR 4.2.1 (Interface):** The application must provide a **clean, intuitive User Interface (UI)** to input requirements and view generated test artifacts.
- **NFR 4.2.2 (Traceability):** All generated test cases must clearly link back to the **original requirement ID** and the **design technique** used.

4.3. Security

- **NFR 4.3.1 (Data Handling):** If requirement data contains sensitive information, the system must implement basic access control (though for a student project, this may be simplified).

4.4. Maintainability and Technology

- **NFR 4.4.1 (Technology Stack):** The application should be developed using a **modern, open-source technology stack** suitable for AI/ML integration (e.g., Python/Flask/Django for backend logic and ML, HTML/CSS/JavaScript or a frontend framework for the UI).
- **NFR 4.4.2 (Modularity):** The architecture must enforce **loose coupling** between the core framework and the individual test design modules (Projects 1-10) to allow for independent development and swapping of algorithms.
- **NFR 4.4.3 (Documentation):** All code must be well-documented, and the project must include a **System Architecture Diagram** and a **User Manual**.

---

5. Technical Constraints

- **C 5.1 (AI Framework):** Students must use common AI/ML libraries (e.g., **Scikit-learn, TensorFlow, PyTorch, or spaCy** for NLP) for their respective modules.
- **C 5.2 (Data):** The system must include a sample mock dataset of requirements and historical execution results for training and demonstration purposes.
- **C 5.3 (Language):** All core logic and UI must be developed in the chosen primary development language (e.g., Python).